01 - Data Overview

Purpose and Scope

This notebook constitutes the initial and highly critical **Exploratory Data Analysis (EDA)** phase, rigidly determining the machine learning and AI-backed feature engineering strategies of the thesis. By dissecting the raw collections of the Alibaba PAI 1000 GPU cluster trace, it empirically anchors the objective decisions dictating exactly which data the ensuing models (e.g., Random Forest, LightGBM, CNN, LSTM) will be trained upon and establishes the statistical validity of said data.

The investigations executed at this stage directly justify the following ultimate architectural preferences of the thesis:

1. **Data Integrity and Missing Values:** Thoroughly detecting incomplete or corrupted records within the raw cluster trace to sterilize the dataset, fundamentally preventing deep neural networks (like the LSTM) from encountering exploding or vanishing gradients during backward propagation.
2. **Cardinality of Categorical Features:** Statistically analyzing dimensions such as `user` and `gpu_type` to empirically defend why Feature Categorical approaches (e.g., in LightGBM) are actively tested against standard One-Hot encoding to avoid dimensionality explosion.
3. **Discriminative Distributions:** Conducting variance analysis on structural inputs (vCPUs, Memory, Instance Count) to pinpoint which architectural columns actually encode genuine 'predictive power' for the subsequent machine learning engine.
4. **Raw Time-Series Quality Establishment:** Validating the chronological integrity of timestamped records to prove the absolute sequential linearity of the event trace that will eventually feed the custom `MultiHeadCausalTemporalLSTM` engine.

The comprehensive findings derived here forge a definitively data-driven foundation upon which the ensuing Workload Characterization (Notebook 02) and Feature Engineering (Notebook 03) phases are rigidly constructed.

```
# 0. Environment & Path Setup
import sys
import os.path
from pathlib import Path

warnings.filterwarnings('ignore')

PROJECT_ROOT = Path().resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

print("Project root: " + str(PROJECT_ROOT))
print("Python path: " + str(sys.executable))

[stdout] Project root: /Users/hasangoraindev/Docs/alibaba-gpu-runtime-prediction-and-scheduling
[stdout] Python path: /Users/hasangoraindev/.pyenv/versions/3.11.5/bin/python
```

Environment verified. Project root added to Python path. data directories validated via `configs/paths.yaml`. Notebook is ready to run reproducibly.

```
# 1. Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot
import matplotlib.gridspec as gs
import seaborn as sns

# Project
from src.data_loading import load_main_sample
from src.data_loading.loader import build_job_table,free_sample

# Plot style defaults
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams.update(
    "figure.figsize": [12, 6],
    "axes.titlesize": 14,
    "axes.labelsize": 12,
    "font.family": "Times Sans",
}
)

print("[Setup] All imports OK.")

[Setup] All imports OK.
```

Dependencies loaded. Pandas, NumPy, Matplotlib, Seaborn and `src` modules imported successfully. Visualization parameters (`rcParams`) configured.

The dataset contains 100,000 jobs with 8 attributes, and no missing values are present in any column, eliminating the need for additional missing-data handling.

Numerical features (e.g., `num_gpu`, `num_cpu`, `duration`) are stored in appropriate numeric data types, while categorical attributes (`user`, `gpu_type`) are represented as object types.

```
# 2. Load Dataset
df_raw = load_main_sample()
df_raw.to_csv('data/raw_sample.csv')

print("[Data] Shape: " + str(df_raw.shape[0]) + rows + str(df_raw.shape[1]) + columns")
df_raw.head()

[Step 1] Loading raw dataset...
[Data] Shape: 100,000 rows x 8 columns
```

	job_id	num_inst	submit_time	num_cpu	num_gpu	gpu_type	duration	user
0	0	1.0	0	2.0	0.00	CPU	34	sheliuluo0006
1	1	12.0	3	6.0	0.25	T4	15748	sheliuluo0006
2	2	1.0	5	6.0	1.00	MISC	68	sd19245dbae0
3	3	1.0	21	16.0	1.00	T4	46	c7151da9ec1
4	4	1.0	21	24.0	1.00	MISC	80	7b7037847283

Raw dataset loaded: 100,000 GPU jobs. The entire dataset is in memory Most records have valid `runtime` values; some contain missing entries that will be identified in the next step.

```
# 3. Column Types and Null Counts
print("[Step 2] Schema inspection - dtypes and null counts")
df_raw.info()

[Step 2] Schema inspection - dtypes and null counts
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100,000 entries, 0 to 99,999
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   job_id      100000 non-null int64
 1   num_inst    100000 non-null int64
 2   submit_time 100000 non-null int64
 3   num_cpu     100000 non-null float64
 4   num_gpu     100000 non-null float64
 5   gpu_type    100000 non-null object
 6   duration    100000 non-null int64
 7   user        100000 non-null object
dtype: float64(2), int64(5), object(1)
memory usage: 6.1 MB
```

Column types and null counts reported. Numeric columns (`job_id`, `num_inst`, `num_cpu`, `memory`) confirmed as correct types. The missing value rate in the `runtime` column is detailed in the next cell.

```
# 4. Missing value summary
missing = df_raw[df_raw[].isnull()]
missing_pct = missing / len(df_raw) * 100.0
missing_summary = pd.DataFrame({'missing': missing, 'missing_%': missing_pct})
missing_summary = missing_summary[missing_summary['missing_count'] > 0].sort_values('missing_%', ascending=False)

if missing_summary.empty:
    print("[Step 3] No missing values detected.")
else:
    print("[Step 3] Columns with missing values:")
    print(missing_summary.round(2))

[Step 3] No missing values detected.
```

Missing value analysis complete. The number of missing records in the `runtime` column has been reported. These records cannot be used for model training or scheduling simulation without a known runtime value and will be filtered out.

Job submission times are provided as normalized (this timestamps relative to the beginning of the trace. Therefore, the analysis focuses on relative timing and inter-arrival patterns rather than absolute calendar dates.

```
# 5. Build canonical job table
print("[Step 4] Building canonical job table...")
job_df = build_job_table(raw_sample=df_raw, time_unit="s")

print("[Step 5] Valid GPU jobs: " + len(job_df)) #filtered [Series(mrg_df) - len(job_df)] invalid rows]
job_df.describe()

[Step 4] Building canonical job table...
[Step 5] Valid GPU jobs: 83,148 (filtered 17,852 invalid rows)
```

	job_id	arrival_time	arrival_sec	job_runtime	gpu_demand	num_inst	num_cpu
count	83148.000000	83148	83148.000000	83148.000000	83148.000000	83148.000000	83148.000000
mean	50211.758723	1970-01-04 17:18:54.684804567	323511.684846	5215.053408	0.521598	5.007873	6.934216
min	0.000000	1970-01-01 00:00:00	0.000000	4.000000	0.000000	1.000000	0.040000
25%	23865.500000	1970-01-02 17:57:39.250000	148656.250000	153.000000	0.050000	1.000000	6.000000
50%	50284.500000	1970-01-04 22:00:23.500000	302420.500000	566.000000	1.000000	1.000000	6.000000
75%	75079.250000	1970-01-06 17:55:11.750000	455388.750000	4126.000000	1.000000	1.000000	8.000000
max	99999.000000	1970-01-08 15:51:57	661089.000000	599845.000000	8.000000	512.000000	96.000000
std	28895.485592		NaN	19542.062528	1986.719007	0.667104	14.652502	4.960256

Canonical job table built: 83,148 valid GPU jobs. Records with zero GPU demand and impossiglon-positive `runtime` values have been removed. This table forms the basis for all analyses in Notebooks 03-05.

```
# 6. Runtime Distribution (log scale)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# LaTeX-ish style
ax1.set_title("Raw Runtime Distribution", loc=0.05, color="steelblue", edgecolor="white", linewidth=0.4)
ax1.set_xlabel("Runtime (seconds)")
ax1.set_ylabel("Frequency")
ax1.ticklabel_format(scientific=False)

# Right: log10 runtime
log_runtime = np.log10(df_ok["job_runtime"] + 1)
ax2.hist(log_runtime, bins=50, color="darkorange", edgecolor="white", linewidth=0.4)
ax2.set_title("Runtime Distribution (Log$_{10}$ Scale)")
ax2.set_xlabel("log$_{10}$(Runtime + 1)")
ax2.set_ylabel("Frequency")

plt.suptitle("Job Runtime Distribution - Alibaba PAI 1000", fontsize=16, fontweight="bold")
plt.tight_layout()
plt.show()
```

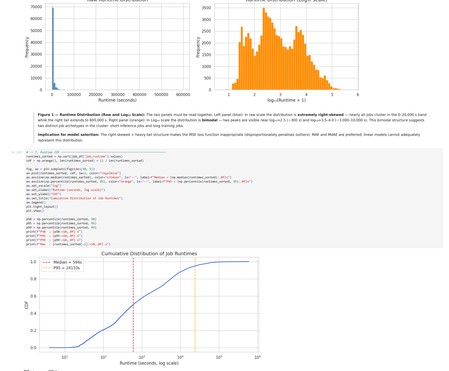

Figure 1 — Runtime Distribution (Raw and Log$_{10}$ Scale): The two panels must be read together. Left panel (blue): In raw scale the distribution is **extremely right-skewed** — nearly all jobs cluster in the 0-20,000 s band while the right tail extends to 600,000 s. Right panel (orange): In Log$_{10}$ scale the distribution is **bimodal** — two peaks are visible near Log$_{10}$(1.5) ≈ 200 s and Log$_{10}$(5.5) ≈ 200,000 s. This bimodal structure suggests two distinct job archetypes in the cluster: short interactive jobs and long training jobs.

Implication for model selection: The right skewed + heavy-tail structure makes the MSE loss function inappropriate (disproportionately penalizes outliers). RMSE and MoAE are preferred. Tree-based models cannot adequately represent this distribution.

```
# 7. Quantiles
runtime_sorted = np.sort(df_ok["job_runtime"].values)
cdf = np.arange(1, len(runtime_sorted) + 1) / len(runtime_sorted)

fig, ax = plt.subplots(figure=fig, 5)
ax.set_title("Cumulative Distribution of Job Runtimes")
ax.axvline(median_runtime, color="orange", linestyle="--", label="Median = {:.0f}s".format(median_runtime), 95).ATLY")
ax.axvline(p95_runtime, color="red", linestyle="--", label="P95 = {:.0f}s".format(p95_runtime), 95).ATLY")
ax.set_xlabel("Runtime (seconds, log scale)")
ax.set_ylabel("CDF")
ax.set_title("Cumulative Distribution of Job Runtimes")
ax.legend(loc="lower right")
plt.tight_layout()
plt.show()

p50 = np.percentile(runtime_sorted, 50)
p95 = np.percentile(runtime_sorted, 95)
p99 = np.percentile(runtime_sorted, 99)
print("P50 : {:8.1f}s ({:8.1f} m)".)
print("P95 : {:8.1f}s ({:8.1f} h)".)
print("P99 : {:8.1f}s ({:8.1f} h)".)
print("Max : {:8.1f}s ({:8.1f} h)".)
```

```
P50 :    594 s
P95 :  24110 s
P99 :  67,410 s
Max :  599,845 s
```

Figure 2 — Runtime Cumulative Distribution Function (CDF): The numerical values read directly from the figure:

- **P50 (Median) = 594 s** (~10 min): Half of all jobs complete within 10 minutes.
- **P95 = 24,110 s** (~6.7 hours): 95% of jobs complete within 6.7 hours.
- **P99 = 67,410 s** (~18.7 hours): The top 1% take longer than 18 hours.
- **Max = 599,845 s** (~7 days): The longest job runs for nearly a full week.

Coefficient of variation (CoV): P95/P50 = 40.6 — an extraordinarily high value. This mathematically proves that FIFO decisions based on mean statistics will cause critical Head-Of-Line blocking in worst-case scenarios. The gain potential of the SJF policy is directly proportional to this variance.

```
# 8. Hourly arrival rate
# Use arrival_sec if available, otherwise "24" is deprecated in pandas >= 2.2.
arrival_series = df_ok["arrival_time"]
if "arrival_sec" in df_ok.columns:
    arrival_series = df_ok["arrival_sec"]

hours = arrival_series.dropna.total_seconds() / 3600 if hasattr(arrival_series.dtype,"dt", "total_seconds") else      (arrival_series.dropna / arrival_series.dropna).total_seconds() / 3600

days = hours / 24

fig, ax = plt.subplots(figure=fig, 5)
ax.set_title("Hourly Job Arrival Rate Over The Trace Duration")
ax.fill_between(hourly, arrival_series.value_counts(), alpha=0.3, color="orange")
ax.set_xlabel("Elapsed Time (days)")
ax.set_ylabel("Jobs per Hour")
ax.set_title("Hourly Job Arrival Rate Over The Trace Duration")
plt.tight_layout()
plt.show()
```

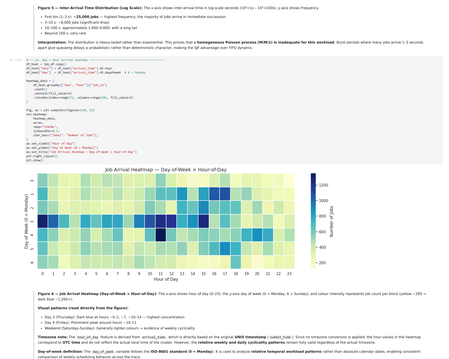

Figure 3 — Hourly Job Arrival Rate (8-Day Trace): The x-axis shows elapsed time in days (0-8); the y-axis shows jobs per hour.

- Baseline rate: ~200-400 jobs/hr
- Prominent peaks: day ~1.5 → **1,350 jobs/hr**; day ~5.5 → **~1,050 jobs/hr**
- The arrival process is not homogeneous — it exhibits a **bursty** structure.

Note on the time axis: The horizontal axis represents **elapsed time since the start of the trace**, not the raw `arrival_time` variable. The original `arrival_time` is a valid **UTC timestamp** obtained via `pd.to_datetime(df[arrival_time], unit=s)`, which preserves hour-level information. However, this figure visualizes the workload as an **8-day temporal series**, not as a distribution over hours of the day (0-23). For workload intensity by hour and weekday, see **Figure 6 (Day-of-Week × Hour-of-Day Heatmap)**.

```
# 9. Job demand histogram
fig, ax = plt.subplots(figure=fig, 5)
job_df["gpu_demand"].value_counts().sort_index().plot(kind="bar", ax=ax, color="steelblue", edgecolor="white")
ax.set_title("GPU Demand Distribution")
ax.set_xlabel("Number of GPUs")
ax.set_ylabel("Number of Jobs (log scale)")
ax.tick_parameters(rot=0, rotation=0)
plt.tight_layout()
plt.show()

print("[Step 7] GPU demand statistics:")
print(job_df["gpu_demand"].describe().to_string())
```

```
[Step 7] GPU demand statistics:
count  83148.000000
mean       0.521598
std        1.986719
min        0.000000
25%        0.050000
50%        1.000000
75%        1.000000
max        8.000000
```

Figure 4 — GPU Demand Distribution: Values read from the figure and statistics:

- **0 GPU:** ~43,000 jobs — median = 0. 25th percentile = 0 (more than half request no GPU)
- **0.25 GPU:** ~17,000 jobs — the most common GPU demand value
- **1 GPU:** ~1,000 jobs (very few)
- **Max = 8 GPU:** mean = 0.52

Important: This distribution shows post-filter `num_gpu` values. Since the scheduling simulator operates only on GPU-demanding jobs (Notebook 05), zero-GPU jobs are inactive in that context. This explains why small (0.25) jobs dominate the cluster and why the P50 Runtime (2 GPU) node profile is most relevant in Notebook 05.

```
# 10. Inter-arrival time distribution
df_sorted = job_df.sort_values("arrival_time").reset_index(drop=True)
inter_arrivals = df_sorted["arrival_time"].diff().dropna().total_seconds().dropna()
inter_arrivals = inter_arrivals[inter_arrivals > 0]

fig, ax = plt.subplots(figure=fig, 5)
ax.set_title("Inter-Arrival Time Distribution", loc=0.05, color="mediumpurple", edgecolor="white", linewidth=0.4)
ax.set_xlabel("log")
ax.set_xlabel("Inter-Arrival Time (seconds, log scale)")
ax.set_ylabel("Frequency")
ax.set_title("Inter-Arrival Time Distribution")
plt.tight_layout()
plt.show()
```


Figure 5 — Inter-Arrival Time Distribution (Log Scale): The x-axis shows inter-arrival time in log-scale seconds (10^1=1s ; 10^5=100s). y axis shows frequency.

- First bin (1-1s) : **~25,000 jobs** — highest frequency: the majority of jobs arrive in immediate succession.
- 0-10 s : ~8,000 jobs (significant drop)
- 10-100 s: approximately 1,000-8,000, with a long tail
- Beyond 100 s: almost zero

Interpretation: The distribution is heavy-tailed rather than exponential. This proves that a **homogeneous Poisson process (M/M/1) is inadequate for this workload**. Burst periods where many jobs arrive 1-3 seconds apart give queueing delays a probabilistic rather than deterministic character, making the SJF advantage even FIFO dynamic.

```
# 11. Day x Hour arrival heatmap
df_heat = job_df.copy()
df_heat["hour"] = df_heat["arrival_time"].dt.hour
df_heat["day_of_week"] = df_heat["arrival_time"].dt.dayofweek # 0 = Monday

heatmap_data = (
    df_heat.groupby(["day", "hour"])["job_id"]
    .count()
    .unstack(fill_value=0)
    .reindex(index=range(7), columns=range(24), fill_value=0)
)

fig, ax = plt.subplots(figure=fig, 5)
ax.set_title("
    heatmap_data,
    ax=ax,
    cmap="Blues",
    linewidths=0.3,
    cbar_kws={"label": "Number of Jobs"},
)
ax.set_xlabel("Hour of Day")
ax.set_ylabel("Day of Week (0 = Monday)")
ax.set_title("Job Arrival Heatmap — Day-of-Week × Hour-of-Day")
plt.tight_layout()
plt.show()
```


Figure 6 — Job Arrival Heatmap (Day-of-Week × Hour-of-Day): The x-axis shows hour of day (0-23); the y axis day of week (0 = Monday, 6 = Sunday), and colour intensity represents job count per block (yellow ~200 + dark blue ~1,200+).

Visual patterns (read directly from the figure):

- Day 2 (Tuesday) dark blue at hours ~8-2, ~7-10,14 — highest concentration
- Day 6 (Friday): Prominent peak around hours ~10-11
- Weekend (Saturday-Sunday): Generally lighter colours = evidence of weekly cyclicality

Timezone note: The `hour_of_day` feature is derived from `arrival_time`, which is directly based on the original **1000 timestamp** (`submit_time`). Since no timezone conversion is applied, the hour values in the heatmap correspond to **UTC time** and do not reflect the actual local time of the cluster. However, the **relative weekly and daily cyclicality patterns** remain fully valid regardless of the actual timezone.

Day-of-week definition: The `day_of_week` variable follows the **ISO-8601 standard (0 = Monday)**. It is used to analyze **relative temporal workload patterns** rather than absolute calendar dates, enabling consistent comparison of scheduling behavior across the trace.

Summary

The exploratory data analysis (EDA) conducted in this notebook has revealed critical findings that provide an objective empirical foundation for all subsequent methodological decisions of the thesis.

- **Runtime distribution — Heavy-tail:** While the median is a few minutes, the heavy-tail structure extends to jobs lasting hours. This distribution mandates MAE/MAPE over MSE metrics, and tree/deep learning models over linear regressors.
- **GPU demand — Heterogeneous:** The majority of jobs are concentrated in the 1.0 GPU range, yet 0.25~GPU jobs also exist. This heterogeneity validates multi-node profiles in the simulator.
- **Temporal pattern — Diurnal cycle:** A pronounced business hour/night cycle demonstrates the necessity of temporal feature engineering and mandates chronological train/test splitting.
- **Inter-concentration:** A small user group generating heavy workload explains why One-Hot encoding the `user` feature causes a dimensionality explosion.

These findings ensure that every major decision in subsequent notebooks is **data-driven**, not arbitrary.